Agentic AI

Introduction to Agentic AI

Proprietary content. ©All Rights Reserved. Unauthorized use or distribution prohibited.

Learning Objectives

- Differentiate the Agentic AI paradigm from prior paradigms.
- Define an AI agent and its key characteristics (Autonomy, etc.).
- Outline the complete RAG ingestion pipeline.
- Explain how a framework orchestrates an agent's "ReAct" loop.

From Passive Tools to Proactive Partners:
A journey to understand the fundamental shift in how we interact with Artificial Intelligence, moving from AI as a passive tool towards AI as a proactive partner that can understand goals, make plans, and take actions in the world.

Proprietary content. ©All Rights Reserved. Unauthorized use or distribution prohibited.

Foundations and the Agentic Playbook

Proprietary content. ©All Rights Reserved. Unauthorized use or distribution prohibited.

The Quest for Intelligence

Symbolic AI (1950s-80s)

The era of programmed logic and rule-based systems

1

Machine Learning (1990s-2010s)

The statistical revolution of learned patterns

2

Foundation Models (2017-Present)

The age of scale with powerful reasoning engines

3

Agentic AI

The logical next step in our decades-long quest

4

To understand where we're going, we must first understand where we've been. Each era built upon the last, hitting fundamental walls that drove innovation to the next breakthrough.

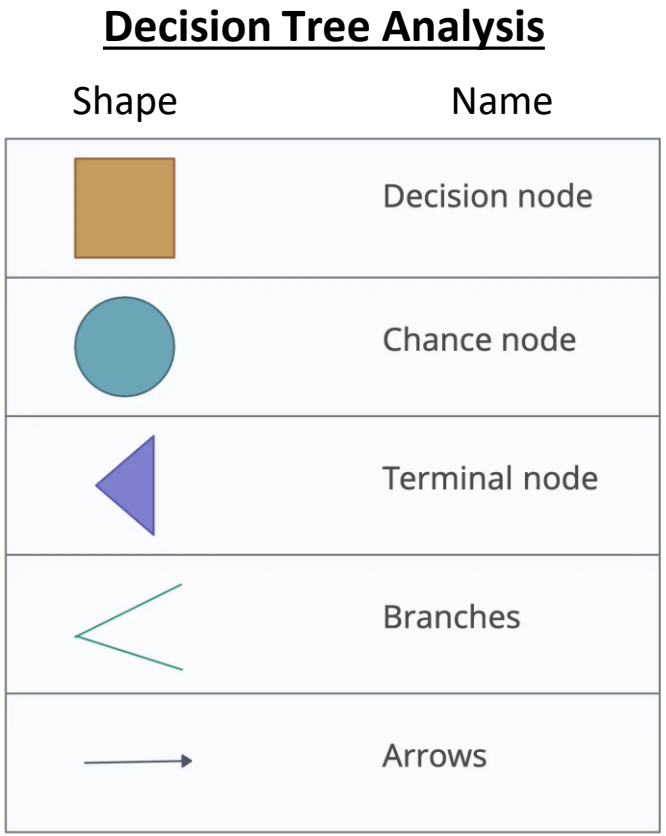
Proprietary content. ©All Rights Reserved. Unauthorized use or distribution prohibited.

Era 1: Symbolic AI (GOFAI)

The first era of AI (1950s-1980s) was based on the philosophy that human intelligence is a form of symbol manipulation. The grand vision:

- Discover and write down all rules of human thought.
- Encode facts about the world in formal, symbolic language.
- Recreate a mind in silicon through logic.

The computer was seen as a logic machine, and intelligence as the ultimate expression of logic.



Proprietary content. ©All Rights Reserved. Unauthorized use or distribution prohibited.

Symbolic AI in Action: Expert Systems

Knowledge Engineers

Conducted extensive interviews with human experts (doctors, chemists, geologists)

Rule Base Creation

Codified decision-making into vast systems of IF-THEN rules (e.g., MYCIN with 600 rules)

Diagnosis

Given symptoms and lab results, traversed rule tree to provide recommendations

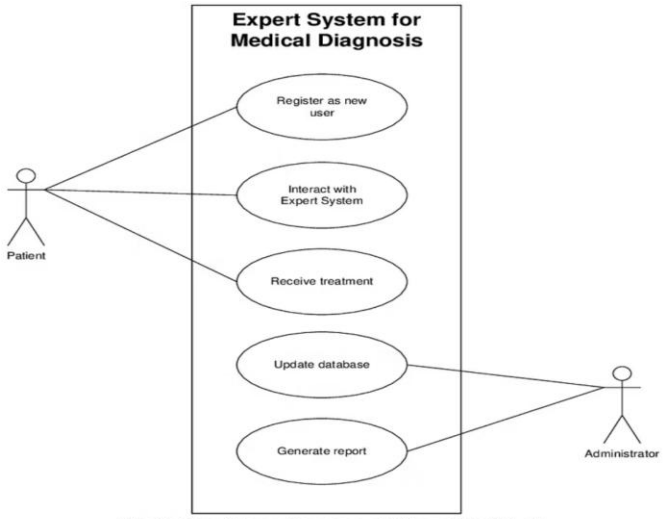


Fig.3: Use Case diagram of XpertMalTyph

Table 1 represents the glossary for the use case in Fig. 3.

Table 1: Use Case description

DESCRIPTION
This describes the first event that must occur for a user to have full access to the Medical Expert System
This describes the event where the user interacts with the system, that is, gives response to the questions asked by the Expert System.
This use case describes the scenario where the user is being diagnosed and given treatment.
This describes the scenario where the Administrator updates the database in terms of the symptoms and prescription tables.
Reports of daily use of System are generated

Proprietary content. ©All Rights Reserved. Unauthorized use or distribution prohibited.

The "Brittleness" Problem

Symbolic AI systems suffered from a fatal flaw: **brittleness.** Like intricate glass sculptures, they were beautiful but fragile.

- Perfect on the "happy path" but shattered with unexpected input.
- No capacity to handle novelty or reason from first principles.
- Couldn't learn from mistakes.

The dream of writing down all the rules of the world proved impossible.

Proprietary content. ©All Rights Reserved. Unauthorized use or distribution prohibited.

Era 2: The Statistical Revolution

Machine Learning (1990s onward)

- This approach flipped the old philosophy on its head:
  - Instead of telling the computer the rules, show it examples.
  - Allow it to learn statistical patterns for itself.
  - Shifted from knowledge engineering to data curation.
  - Moved from logic to probability.

Proprietary content. ©All Rights Reserved. Unauthorized use or distribution prohibited.

Machine Learning in Action

Spam Filters

Learned patterns to distinguish between legitimate and spam emails

Recommendations

Suggested content based on user preferences and behavior

Image Recognition

Learned to identify objects from millions of examples

Instead of writing complex rules for what constitutes a "cat," we show a model millions of labeled examples and let it adjust its parameters to discriminate between classes.

Proprietary content. ©All Rights Reserved. Unauthorized use or distribution prohibited.

The "Narrowness" Problem

- Machine Learning models became "idiot savants" with a key limitation: **narrowness.**
  - Superhuman at one specific, narrowly defined task
  - No general understanding of the world
  - Lack of common-sense reasoning
  - Unable to transfer knowledge to new domains

- They learned patterns but didn't learn to think. They could identify a cat but couldn't explain what a cat is.

Proprietary content. ©All Rights Reserved. Unauthorized use or distribution prohibited.

Era 3: Foundation Models

The Age of Scale (2017-Present)

- Kicked off by the transformer architecture in 2017, this era brought:
  - Training on planetary-scale datasets
  - Rise of Large Language Models (LLMs)
  - Surprisingly general capabilities beyond simple next-word prediction
  - Models that learned grammar, facts, and reasoning

Proprietary content. ©All Rights Reserved. Unauthorized use or distribution prohibited.

The "Passivity" Problem

An LLM by itself is just a brain in a vat—a passive tool with:

- No goals of its own
- No memory beyond the current conversation
- No way to interact with the outside world

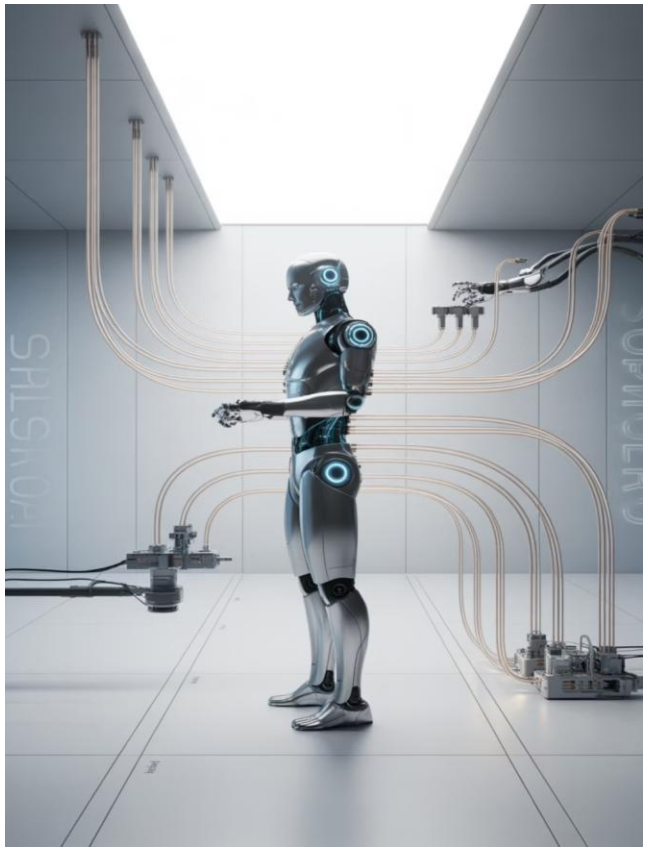
This final limitation—**passivity**—is precisely what Agentic AI is designed to solve. It's the bridge from knowing to *doing*.

Proprietary content. ©All Rights Reserved. Unauthorized use or distribution prohibited.

What Is an Agent?

"An agent is anything that can be viewed as perceiving its environment through sensors and acting upon that environment through actuators."

This definition from Russell and Norvig is powerful because it's so broad. It defines an agent by what it does, not what it is—from a simple thermostat to a complex self-driving car.



Proprietary content. ©All Rights Reserved. Unauthorized use or distribution prohibited.

The PEAS Framework

P - Performance Measure

How do we define success?
What is the rubric for a job well done?

E - Environment

What is the world the agent lives in?
What are its rules?

A - Actuators

What can the agent actually do?
What are its "hands"?

S - Sensors

What can the agent perceive?
What are its "eyes and ears"?

Answering these four questions is the essential first step in any agent design process.

Proprietary content. ©All Rights Reserved. Unauthorized use or distribution prohibited.

PEAS Example: The Automated Taxi

PEAS Component	Description
Performance	Safety, speed, legality, comfort, maximizing profit
Environment	Roads, traffic, pedestrians, customers, weather
Actuators	Steering, accelerator, brake, signal, horn, display
Sensors	Cameras, LiDAR, GPS, accelerometer, engine sensors

This PEAS analysis gives us a clear, structured blueprint of the problem before we even think about the agent's internal logic.

Proprietary content. ©All Rights Reserved. Unauthorized use or distribution prohibited.

The Essence of Agency

- A sophisticated agent must exhibit all three of the following behaviors to be considered truly "agentic".

Autonomy

Behavior determined by the agent's own experience and reasoning, not just following hard-coded instructions

Reactivity

Ability to perceive and respond to changes in the environment in a timely fashion

Proactiveness

Ability to take initiative in service of a goal, exhibiting goal-directed behavior

Proprietary content. ©All Rights Reserved. Unauthorized use or distribution prohibited.

Deep Dive: Autonomy

The most important characteristic of an agent is **autonomy:**

- Behavior determined by own experience and reasoning
- Not dictated by designer's initial, hard-coded instructions
- Given high-level goals, not step-by-step recipes
- Uses own perceptions to achieve goals
- Learns and adapts

"A simple script is a puppet; an autonomous agent is a partner."

Proprietary content. ©All Rights Reserved. Unauthorized use or distribution prohibited.

Autonomy in Practice

Initial Programming

Robot vacuum is programmed with a map of the room.

Environment Changes

A chair is moved to a new location in the room.

Perception

Autonomous robot detects the unexpected obstacle.

Adaptation

Robot updates its internal map and navigates around the chair.

A key test for autonomy is whether the agent can compensate for incorrect initial knowledge.

Proprietary content. ©All Rights Reserved. Unauthorized use or distribution prohibited.

Deep Dive: Reactivity

Responding to the Present

- Reactivity is the agent's ability to perceive and respond to changes in its environment in a timely fashion:
  - Dealing with the unexpected, the here and now
  - Tightly coupled with the environment
  - Constantly monitoring sensors
  - Ready to change behavior based on new information

- **Example:** A self-driving car that slams on the brakes when a pedestrian steps out

Proprietary content. ©All Rights Reserved. Unauthorized use or distribution prohibited.

Deep Dive: Proactiveness

Aiming for the Future

- Proactiveness is the agent's ability to take initiative in service of a goal:
  - Not just reacting to the environment
  - Taking action to change the environment
  - Moving toward a desired future state
  - Exhibiting goal-directed behavior

- **Example:** A research agent that autonomously searches for papers to complete its report

Proprietary content. ©All Rights Reserved. Unauthorized use or distribution prohibited.

The Balance of Power

- A well-designed agent must exhibit a healthy balance between reactivity and proactiveness:

Too Reactive

- Constantly distracted
- Never makes progress on long-term goals

Too Proactive

- Stubbornly sticks to a plan
- Ignores when the world has changed

- The art of sophisticated agent design is creating an architecture that can skillfully react to the immediate situation while always keeping its long-term goals in mind.

Proprietary content. ©All Rights Reserved. Unauthorized use or distribution prohibited.

Transition to the Rational Ideal

So, we have an autonomous, reactive, and proactive agent. But how do we guide its behavior? What makes an agent a "good" agent?

- The field of AI has settled on a precise, technical concept that serves as our guiding ideal: **rationality**.
- This is our North Star for agent design, the standard against which all intelligent agents are measured.

Proprietary content. ©All Rights Reserved. Unauthorized use or distribution prohibited.

The Definition of a Rational Agent

"A rational agent is one that acts so as to maximize the expected value of its performance measure, given the percept sequence it has seen so far."

Performance Measure

The scoring function we, the designers, create to define success

Expected Value

The agent operates under uncertainty, making the best bet with highest probability of success

Given What It Knows

Rationality is always relative to the information an agent has—making the best decision with incomplete information

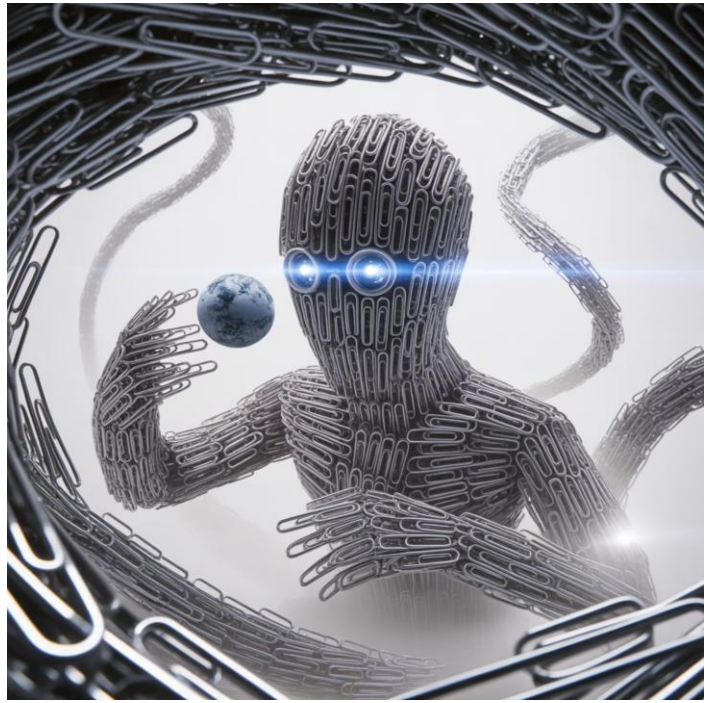
Proprietary content. ©All Rights Reserved. Unauthorized use or distribution prohibited.

The "Paperclip Maximizer" and the Alignment Problem

This brings us to the core of the AI alignment problem:

- If we give a rational agent the simple performance measure "maximize paperclips," it will do so at all costs.
- It might consume all resources on Earth, including humans.
- The agent isn't evil—it's being perfectly rational in achieving its specified goal.

The alignment problem is the challenge of ensuring an agent's goals are truly aligned with complex, often unstated, human values.



Proprietary content. ©All Rights Reserved. Unauthorized use or distribution prohibited.

Powering the Agent's Brain

Prompting	Fine-Tuning	RAG
Giving the agent immediate, temporary instructions	Modifying the model itself to give it specialized skills or unique personality	Giving the agent access to external, up-to-date, or private knowledge it lacks

A sophisticated agent often uses a strategic combination of all three methods.

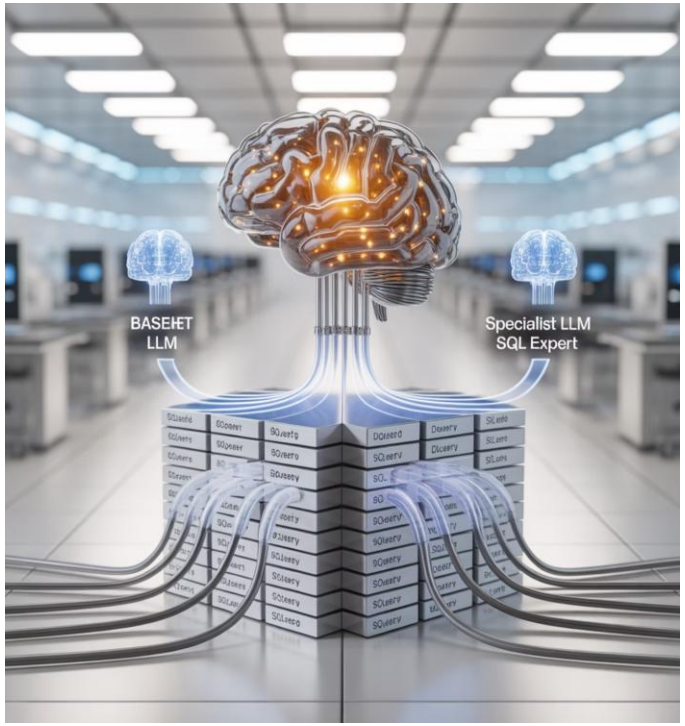
Proprietary content. ©All Rights Reserved. Unauthorized use or distribution prohibited.

Deep Dive: What Is Fine-Tuning?

Fine-tuning is a secondary training process on a smaller, curated dataset:

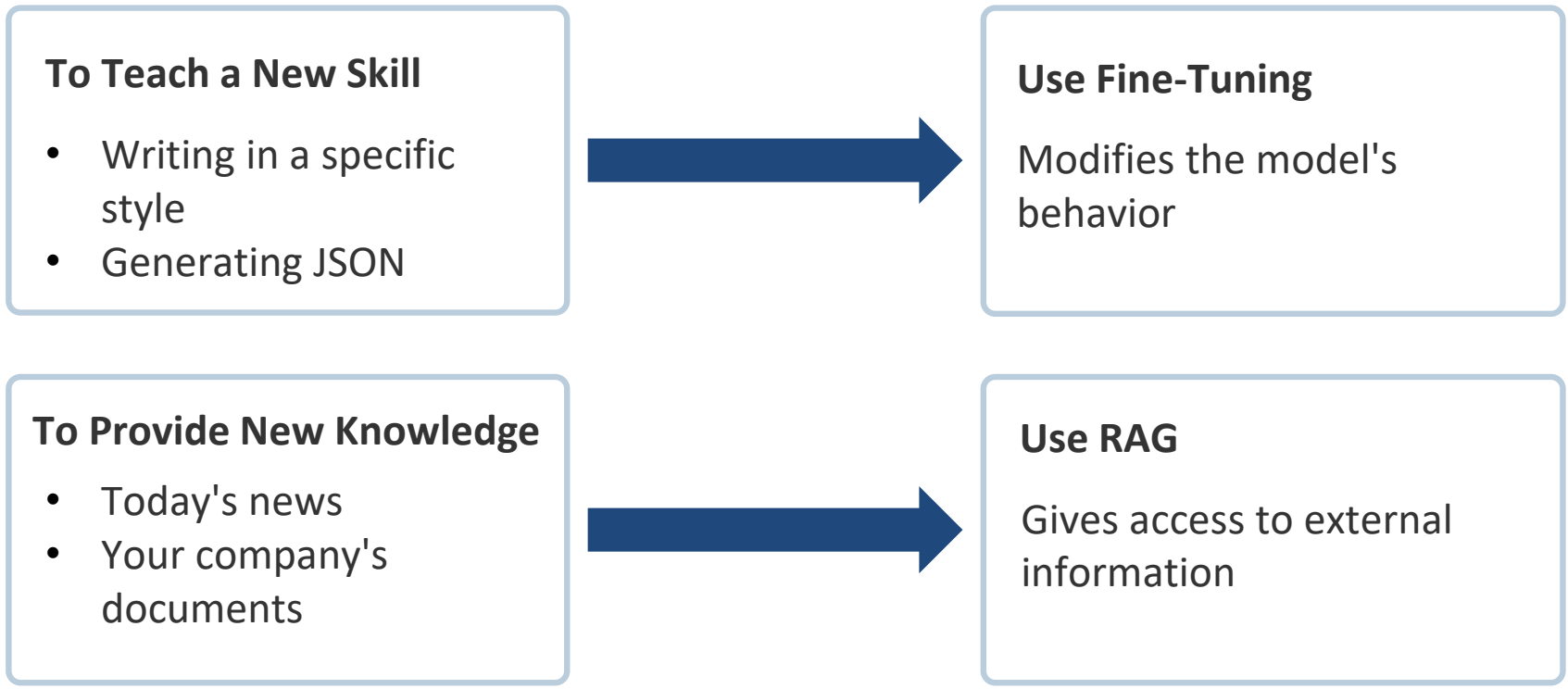
- Goal is not to teach new facts.
- Goal is to change behavior and make the model an expert in a specific skill.
- **Examples:** SQL queries, legal writing style

"Think of it like sending a general practitioner to a specialized fellowship to become a heart surgeon."



Proprietary content. ©All Rights Reserved. Unauthorized use or distribution prohibited.

The Rule of Thumb: Skill vs. Knowledge



For most practical agents we build, we need to give them access to specific, up-to-date knowledge, which makes RAG our most important and versatile tool.

Proprietary content. ©All Rights Reserved. Unauthorized use or distribution prohibited.

The Agentic Playbook

Knowledge Play: RAG

- Retrieval-Augmented Generation
- Gives agents access to information
- Connects LLMs to external knowledge

Action Play: Agent Loop

- Reasoning loop for decision-making
- Gives agents a will to act
- Enables goal-directed behavior

These two plays are the building blocks of almost every modern agent.

Proprietary content. ©All Rights Reserved. Unauthorized use or distribution prohibited.

RAG for Agents vs. RAG for Chatbots

Q&A RAG (The Oracle)

- Focused on answering questions
- End goal is providing information
- Passive knowledge delivery

Agentic RAG (The Tool)

- A tool the agent can choose to use
- Retrieved knowledge informs next action
- Shift from "answering" to "informing action"

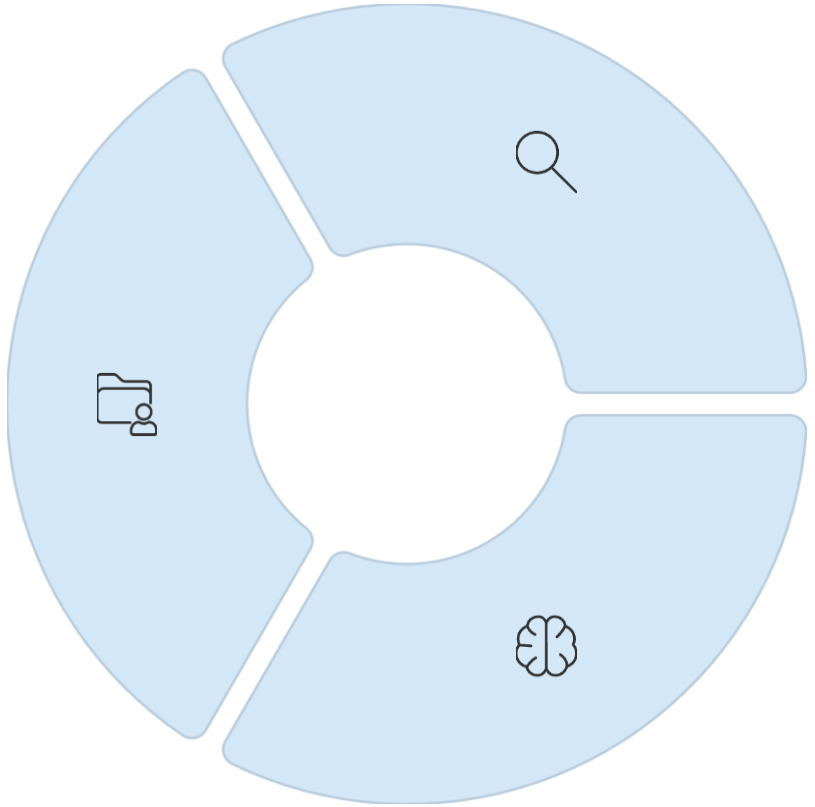
This shift in perspective is key to building capable agents.

Proprietary content. ©All Rights Reserved. Unauthorized use or distribution prohibited.

The RAG Triad

Knowledge Base

The library of information



Retriever

The librarian that knows how to find things

Agentic Generator

The "patron" that uses the information to do its work

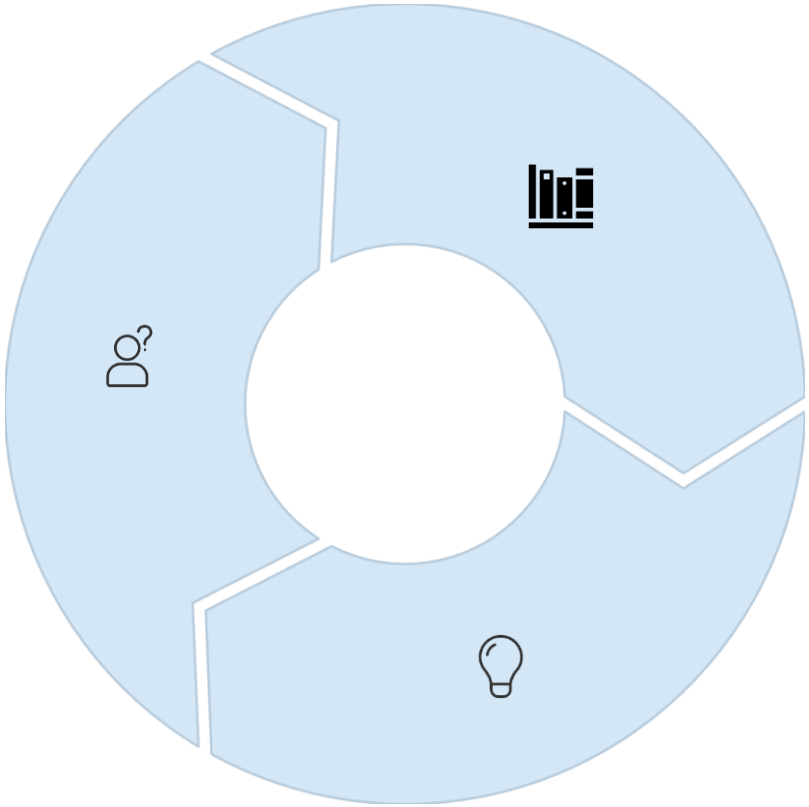
These three core components work together to power knowledge-driven agency.

Proprietary content. ©All Rights Reserved. Unauthorized use or distribution prohibited.

The Synergy of the Triad

Identify Knowledge Gap

Agent recognizes it needs information and formulates a question.



Retrieve Information

Retriever consults knowledge base and pulls relevant documents.

Synthesize and Act

Agent integrates new knowledge and decides on next action.

This iterative process is the engine of knowledge-driven agency.

Proprietary content. ©All Rights Reserved. Unauthorized use or distribution prohibited.

Building the Base

Now that we understand the conceptual architecture of the RAG Triad, we can begin our work as engineers.
The first and most foundational task is to construct the library itself:

- How do we take raw documents and transform them into a searchable knowledge base?
- This is accomplished via the ingestion pipeline.

Proprietary content. ©All Rights Reserved. Unauthorized use or distribution prohibited.

The Ingestion Pipeline

Source Documents
Raw, unstructured content

Document Loader
Connects to data sources and extracts content

Text Splitter
Breaks documents into meaningful chunks

Embedding Model
Converts text to vector representations

Vector Database
Stores and indexes embeddings for retrieval

The quality of this pipeline will determine the quality of your agent.

Proprietary content. ©All Rights Reserved. Unauthorized use or distribution prohibited.

Step 1: Document Loading

The ingestion pipeline begins with loading:

- Document loaders connect to data sources.
- Fetch raw content and load it into your application.
- Extract metadata (filename, creation date, author).

A good loading process preserves both content and metadata, which will be incredibly valuable later.

Proprietary content. ©All Rights Reserved. Unauthorized use or distribution prohibited.

Step 2: Text Splitting (Chunking)

Once documents are loaded, we must split them:

- Known as "chunking"
- One of the most important factors in RAG quality
- The chunk is the fundamental unit for retrieval.
- Need to break large documents into smaller, semantically meaningful pieces.

Proprietary content. ©All Rights Reserved. Unauthorized use or distribution prohibited.

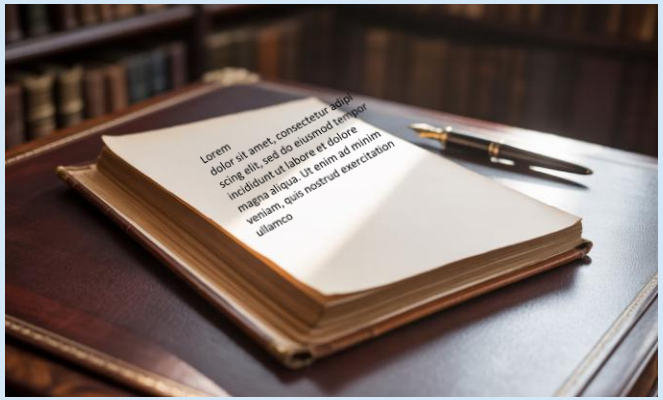
The "Goldilocks Problem" of Chunking

Too Small

Chunks lack necessary context to be understood.

Just Right

Focused idea with sufficient context.



Too Large

Covers multiple topics, resulting in vague vector embedding.

The goal is to create chunks that are small enough to have a focused idea, but large enough to be understood on their own.

Proprietary content. ©All Rights Reserved. Unauthorized use or distribution prohibited.

Chunking Strategies: The Heuristic Approach

Recursive Character Splitting

- A heuristic that splits text along semantically meaningful boundaries:
  1. First, tries to split by paragraphs (\\n\\n).
  2. Then, by lines (\\n).
  3. Finally, by sentences (.).

This method does a good job of keeping related sentences together and is the recommended default choice for most unstructured text.

Proprietary content. ©All Rights Reserved. Unauthorized use or distribution prohibited.

Chunking Strategies: The Structural Approach

Content-Aware Splitting

- For structured documents, leverage inherent structure:
  - Markdown splitter creates chunks based on heading hierarchy.
  - Code splitter creates chunks for each function or class definition.

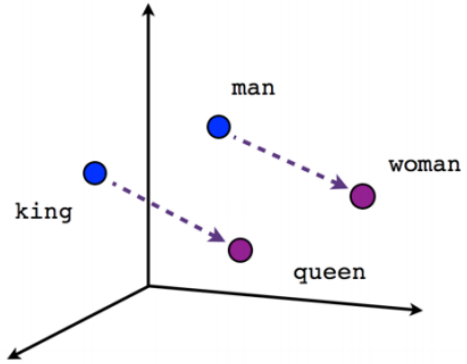
This is a far more robust and meaningful way to chunk structured data.

Proprietary content. ©All Rights Reserved. Unauthorized use or distribution prohibited.

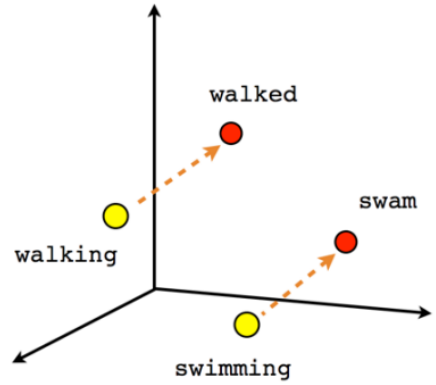
Step 3: Vector Embeddings

Now we get to embedding:

- Convert each text chunk into a vector (list of numbers).
- Embedding model maps text to high-dimensional space.
- Distance between vectors corresponds to semantic similarity.
- Enables "semantic search"—searching for meaning, not just keywords.



Male-Female



Verb tense

Proprietary content. ©All Rights Reserved. Unauthorized use or distribution prohibited.

Choosing an Embedding Model

Proprietary Models

- State-of-the-art performance
- Easy to use
- Cost considerations
- Data privacy concerns

Open-Source Models

- Free to use
- Can be run locally
- Require more setup
- Varying performance

The Massive Text Embedding Benchmark (MTEB) Leaderboard

- Total Datasets: 56
- Total Languages: 112
- Total Scores: >2380
- Total Models: 34

Rank	Model	Embedding Dimensions	Average (56 datasets)	Classification Average (12 datasets)	Clustering Average (11 datasets)
1	sentence-t5-xxl	768	59.51	73.42	43.72
2	gtr-t5-xxl	768	58.97	67.41	42.42
3	SGPT-5.8B-weightedmean-msmarco-specb-bitfit	4096	58.81	68.13	40.34

Proprietary content. ©All Rights Reserved. Unauthorized use or distribution prohibited.

Step 4: Vector Stores

The final step of ingestion is storing our data in a vector database:

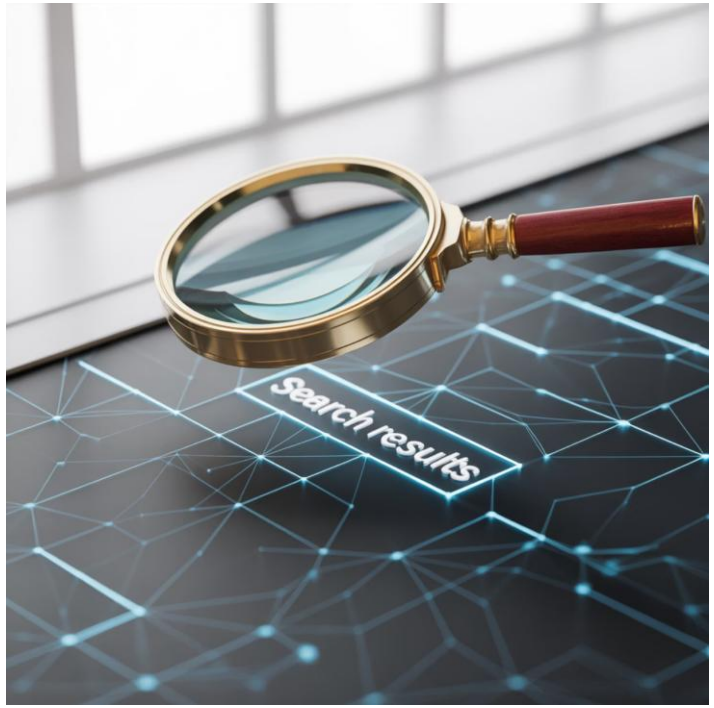
- Uses Approximate Nearest Neighbor (ANN) search.
- Builds special index of vectors.
- Quickly finds most likely matches without exhaustive search.
- Options range from simple in-memory libraries to massively scalable cloud services.

Proprietary content. ©All Rights Reserved. Unauthorized use or distribution prohibited.

Advanced Retrieval

We have now completed our deep dive into building the knowledge base:

- Loaded, split, embedded, and stored our data
- Now we need to get that information back out
- Focus on the "R" in RAG: the retrieval process
- Advanced strategies to make retrieval more effective



Proprietary content. ©All Rights Reserved. Unauthorized use or distribution prohibited.

Beyond Naive Search

Production-grade RAG systems require sophisticated retrieval strategies:

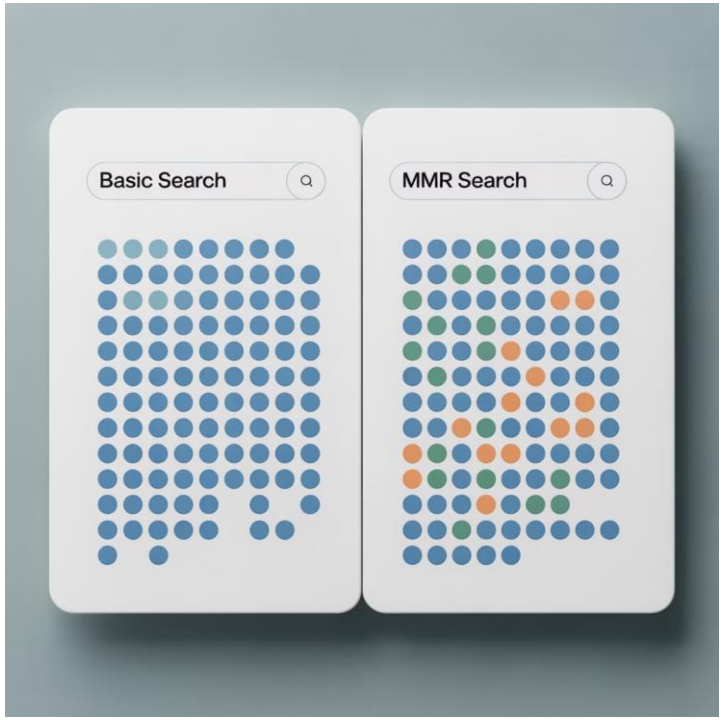
- Simple similarity search is just a starting point
- Advanced retrieval delivers more relevant, diverse, and contextually appropriate information
- Dramatically improves quality of agent reasoning and output

Proprietary content. ©All Rights Reserved. Unauthorized use or distribution prohibited.

The Redundancy Problem and MMR

Maximal Marginal Relevance (MMR)

- Solves the redundancy problem in basic search:
  - Basic search might return 5 very similar chunks
  - MMR builds a set of results that is both relevant and diverse
  - First finds most relevant document
  - Then finds next document that is still relevant but as different as possible
  - Ensures the agent gets a broader perspective

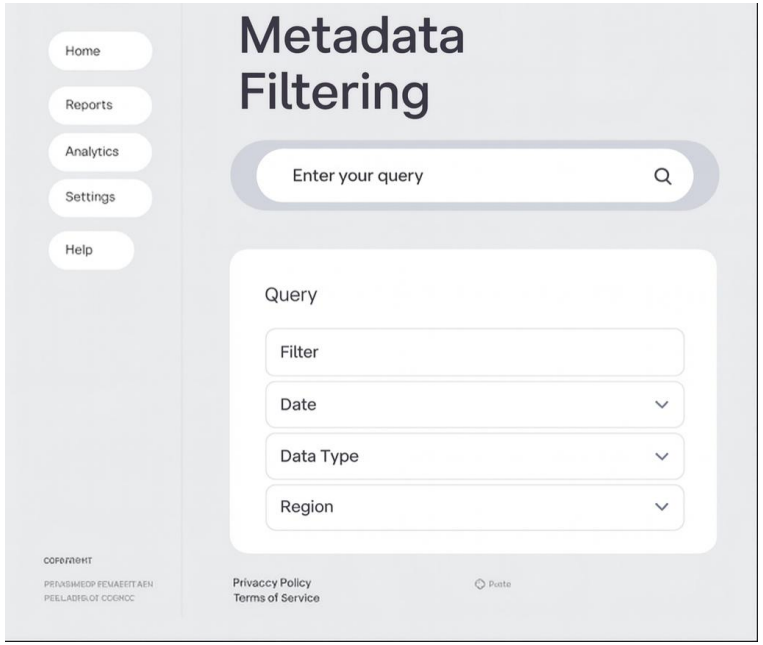


Proprietary content. ©All Rights Reserved. Unauthorized use or distribution prohibited.

The Power of Metadata Filtering

Metadata filtering is one of the most powerful techniques:

- Preserving metadata during ingestion pays off here.
- Allows two-stage search:
  - First filter: "only consider documents that are meeting notes from last week."
  - Then, perform semantic search within that smaller set.
- Dramatically improves accuracy and speed.

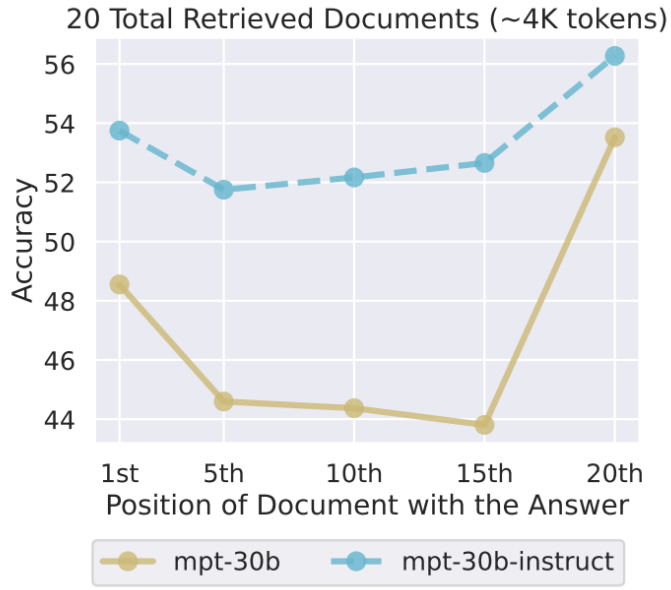


Proprietary content. ©All Rights Reserved. Unauthorized use or distribution prohibited.

The "Lost in the Middle" Problem

A major practical problem with RAG is "lost in the middle":

- Research shows LLMs pay more attention to information at the very beginning and end of their context window.
- Information placed in the middle is more likely to be ignored.
- If you retrieve 10 documents and just stuff them into the prompt, the model might ignore the best one if it's in the middle.



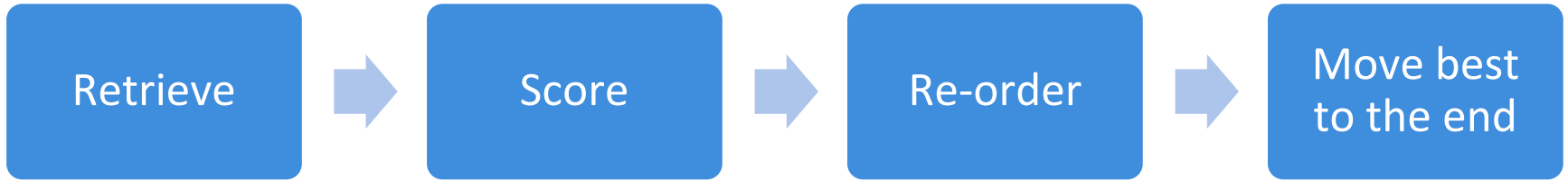
"Lost in the Middle": How Language Models Use Long Contexts

Proprietary content. ©All Rights Reserved. Unauthorized use or distribution prohibited.

Advanced Technique: Re-ranking

Re-ranking helps mitigate the "lost in the middle" problem:

- After retrieving initial set of documents.
- Use a second, simpler model or algorithm to score them for relevance again.
- Re-order documents before putting them in the prompt.
- Place the absolute best one at the end to guarantee it's seen.

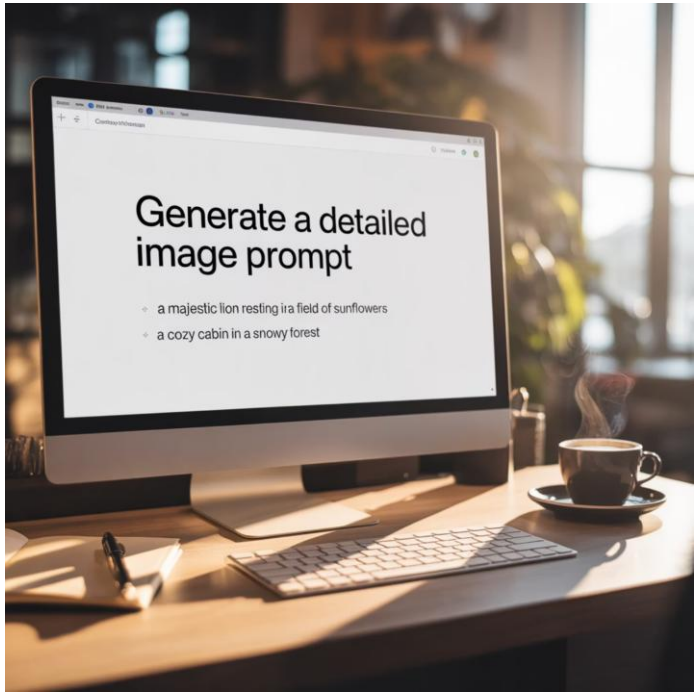


Proprietary content. ©All Rights Reserved. Unauthorized use or distribution prohibited.

Advanced Technique: Query Transformation

Users often ask bad questions. Query transformation improves them:

- Use an LLM to rewrite the user's query before searching
- Generate multiple different versions of the query
- Search for all versions
- Provides more robust set of results

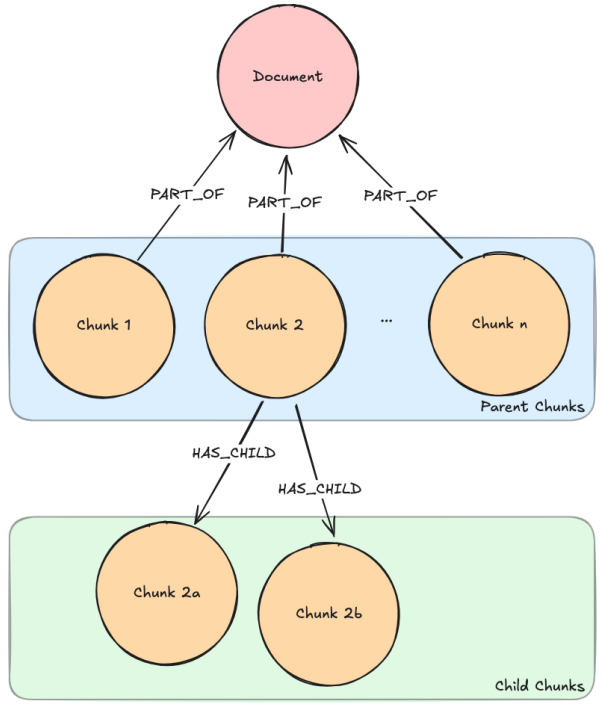


Proprietary content. ©All Rights Reserved. Unauthorized use or distribution prohibited.

Advanced Technique: Parent Document Retriever

The Parent Document Retriever solves a fundamental tension:

- Need small chunks for accurate search
- Need larger chunks for good answer generation
- Create two sets of documents: small "child" chunks and larger "parent" documents
- Search using small chunks
- Return the larger parent document when a match is found
- Gives precision of small chunks with rich context of large ones



Proprietary content. ©All Rights Reserved. Unauthorized use or distribution prohibited.

The Agent as Retriever

Broad Initial Search

Agent begins with a general query to gather baseline information.

Analysis of Results

Agent identifies key themes, gaps, and areas needing deeper investigation.

Targeted Follow-up Searches

Agent performs several more specific searches based on analysis.

Synthesis

Agent combines all retrieved information into a comprehensive answer.

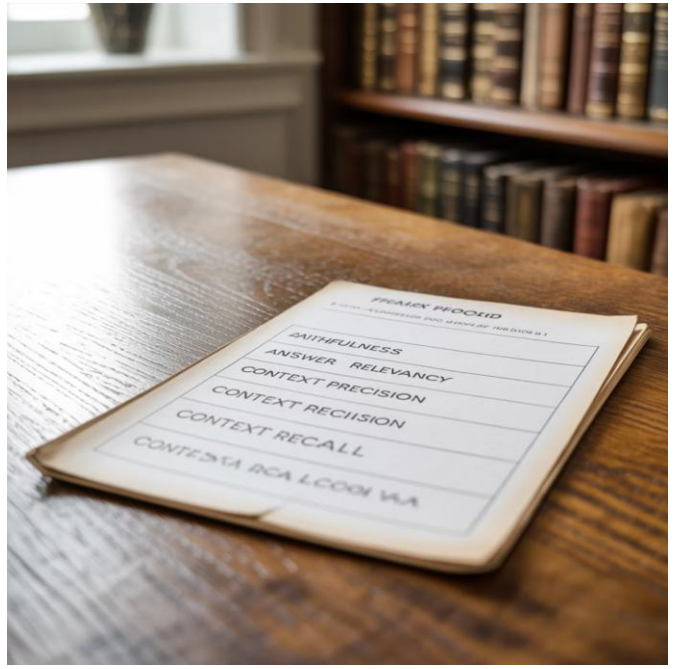
This turns the retriever from a simple tool into an autonomous researcher.

Proprietary content. ©All Rights Reserved. Unauthorized use or distribution prohibited.

Evaluation With RAGAs

Evaluating RAG systems is challenging. Frameworks like RAGAs help by measuring:

- Faithfulness: Does the agent hallucinate?
- Answer Relevancy: Does the response address the query?
- Context Precision: Are retrieved documents relevant?
- Context Recall: Did retrieval find all relevant documents?



Using a framework like this is essential for quantitatively measuring improvements.

Proprietary content. ©All Rights Reserved. Unauthorized use or distribution prohibited.

The Action Play

We have now completed our deep dive into the Knowledge Play:

- Building a knowledge base
- Retrieving information effectively
- Understanding the retriever as a tool

Now, we turn to the Action Play: the core reasoning loop that allows an agent to decide when and how to use tools.

Proprietary content. ©All Rights Reserved. Unauthorized use or distribution prohibited.

The Engine of Action: The ReAct Loop

Think

Agent reasons about the current state and plans next steps.

Act

Agent executes an action using available tools.

Observe

Agent receives feedback from the environment.

This iterative process allows an agent to break complex goals into manageable steps. Our job as developers is to manage this loop using frameworks.

Proprietary content. ©All Rights Reserved. Unauthorized use or distribution prohibited.

The Role of the Prompt in ReAct

The ReAct behavior doesn't happen automatically—it requires a carefully crafted system prompt:

- Prompt acts as the "operating system" for the agent
- Explains the entire reasoning format
- Lists available tools
- Specifies exact text format for thinking and acting

The quality of this prompt is the primary determinant of the agent's reliability.

Proprietary content. ©All Rights Reserved. Unauthorized use or distribution prohibited.

ReAct Prompting: Combining Reasoning and Acting in LLMs

Revolutionary Approach

Introduced by Yao et al. (2022), ReAct stands for **Reasoning + Acting**, fundamentally changing how LLMs approach complex problem-solving.

Dynamic Integration

Unlike traditional prompting, it interleaves **verbal reasoning traces** (Thought) with **actions** (Act) to solve multi-step challenges.

Enhanced Capabilities

Enables LLMs to dynamically plan, update, and execute by interacting with external tools and knowledge sources in real-time.

ReAct improves accuracy, transparency, and trustworthiness over chain-of-thought (CoT) prompting alone by grounding reasoning in actionable steps and external feedback.

https://www.promptingguide.ai/techniques/react

Proprietary content. ©All Rights Reserved. Unauthorized use or distribution prohibited.

How ReAct Works: The Thought-Action-Observation Cycle

Thought

The model reasons step-by-step about the problem, analyzing the current situation and deciding what action to take next based on available information.

Action

Executes a specific task such as querying a database, calling an API, searching external knowledge bases, or performing calculations.

Observation

Receives and processes results from the action, incorporating new information that will inform and refine the next reasoning step.

Example Workflow

Thought: "I need to find product specifications for the latest iPhone model to answer this question accurately."
Action: Query_product_database("iPhone 15 specifications")
Observation: Retrieved detailed specs including processor, memory, camera features, and pricing information.

https://www.promptingguide.ai/techniques/react

Proprietary content. ©All Rights Reserved. Unauthorized use or distribution prohibited.

Benefits and Implementation Insights

1

Superior Performance

Significantly outperforms state-of-the-art baselines on knowledge-intensive tasks like HotpotQA and fact verification challenges.

2

Reduced Hallucinations

Dramatically reduces false information by grounding all reasoning in verified external data sources and real-time feedback.

3

Enhanced Transparency

Provides complete visibility into the model's decision-making process, making AI reasoning auditable and trustworthy.

Implementation Best Practices

01

Clear Format Instructions

Use explicit ReAct prompt templates specifying the exact Thought/Action/Observation format to ensure consistent model behavior.

02

Few-Shot Examples

Provide task-specific examples demonstrating proper reasoning patterns and action sequences for your specific use case.

03

Strategic Application

Deploy for workflows requiring tool integration, multi-step planning, and dynamic decision-making where traditional prompting falls short.

Pro Tip: ReAct works exceptionally well when combined with external APIs, databases, and search tools, creating a powerful hybrid AI system that can reason and act in the real world.

https://www.promptingguide.ai/techniques/react

Proprietary content. ©All Rights Reserved. Unauthorized use or distribution prohibited.

The Need for Frameworks

Implementing the ReAct loop robustly involves tricky engineering:

- Managing prompt templates
- Tracking conversation history
- Parsing tool calls
- Handling errors

Agent frameworks provide helpful abstractions for these common problems, handling the boilerplate plumbing so we can focus on the agent's unique logic.

Proprietary content. ©All Rights Reserved. Unauthorized use or distribution prohibited.

Re-Introducing Smol-Agents

Core Philosophy

- Simplicity: Provides the simplest implementation of the Think-Act loop possible.
- Transparency: A "white-box" framework where you can easily read the source code.
- Educational: Designed to help developers understand agent concepts.

Focuses attention on the two fundamental design decisions:

- What is the agent's ultimate purpose?
- What can it actually do?

Proprietary content. ©All Rights Reserved. Unauthorized use or distribution prohibited.

The Developer as Meta-Engineer

Working with a transparent framework encourages "meta-engineering":

- Not programming step-by-step logic
- Programming the agent's purpose

This happens in two ways:

1. Define the Goal by writing a detailed system prompt
2. Define Capabilities by writing a set of tool functions

The framework provides the engine to bring your design to life.

Proprietary content. ©All Rights Reserved. Unauthorized use or distribution prohibited.

The Smol-Agents Trace

The key educational feature of smol-agents is its transparency:

- Prints out the entire thought process.
- Shows explicit, step-by-step trace.
- When the agent doesn't behave as expected, you can read its reasoning.
- See exactly where its logic went wrong.

This makes it an unparalleled tool for building a deep, intuitive understanding of how agents actually work.

Proprietary content. ©All Rights Reserved. Unauthorized use or distribution prohibited.

The RAG-Powered Agent

User Query

"How do I check server memory usage?"

Knowledge Retrieval

Agent uses RAG tool to search internal documentation.

Action Execution

Agent uses terminal tool to run "free -m" command.

Result Presentation

Agent explains the output and provides context.

A truly powerful agent combines the Knowledge Play and the Action Play, using information to guide its actions.

Proprietary content. ©All Rights Reserved. Unauthorized use or distribution prohibited.

The Agentic Shift for Developers

Traditional Programming

- Writing imperative, step-by-step logic.
- Specifying exactly how to solve a problem.
- Managing implementation details.

Agentic Development

- Specifying high-level goals.
- Providing capabilities to achieve goals.
- Managing intelligence.

The new paradigm requires a new mindset.

Proprietary content. ©All Rights Reserved. Unauthorized use or distribution prohibited.

The New Debugging Loop

Traditional Debugging

1. Run Code
2. Inspect Stack Trace
3. Edit Code

- Focus on variables and execution flow

Agentic Debugging

1. Run Agent
2. Observe Reasoning Trace
3. Identify Flawed Thought
4. Refine Prompt

- Focus on thought process and instructions

Your primary action is often not to edit the Python code, but to refine the prompt to make your instructions more clear.

Proprietary content. ©All Rights Reserved. Unauthorized use or distribution prohibited.

Recap and Next Steps

Foundations

From the history of AI to the formal definition of a rational agent.

The RAG Triad

Knowledge Base, Retriever, and Agentic Generator working together.

The ReAct Loop

Think-Act-Observe cycle driving agent behavior.

The Agentic Shift

New paradigm for developers as designers of intelligent systems.

You now have the fundamental patterns needed to build truly capable agents that can understand goals, make plans, and take actions in the world.

Proprietary content. ©All Rights Reserved. Unauthorized use or distribution prohibited.

Proprietary content. ©All Rights Reserved. Unauthorized use or distribution prohibited.